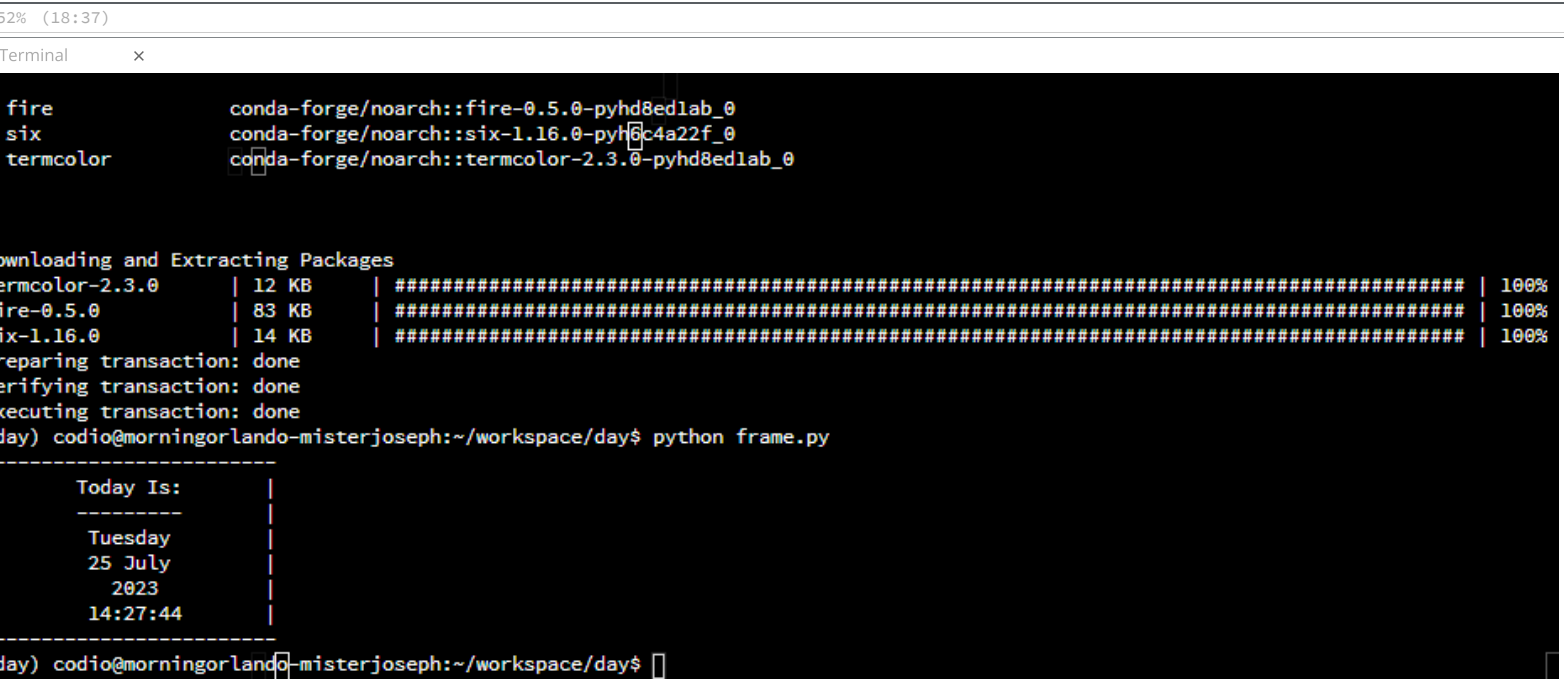


```
frame.py
1 import date
2 import fire
3
4 def main(days='today'):
5     width = 25
6     line = width * '-'
7     info = date.get_info(day)
8     print(line)
9     display_date(day, width, info)
10    print(line)
11
12    def display_date(day, width, info):
13        display_title(day, width)
14        center_text(width, info['day'])
15        center_text(width, f'{info["date"]} {info["month"]}')
16        center_text(width, info['year'])
17        if day == 'today':
18            center_text(width, info['time'])
19
20    def display_title(day, width):
21        if day == 'tomorrow':
22            title = 'Tomorrow Will Be:'
23        elif day == 'yesterday':
24            title = 'Yesterday Was:'
25        else:
26            title = 'Today Is:'
27        line = len(title) * '-'
28        center_text(width, title)
29        center_text(width, line)
```



## Building the Frame Module

### Installing the Fire Package

We are going to install the Fire package, which simplifies the creation of a command line interface. Start by activating the `day` virtual environment. Enter the following command in the terminal.

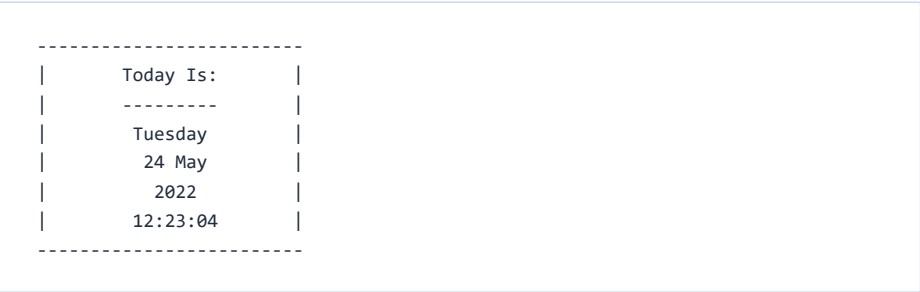
```
conda activate day
```

Then install Fire and its dependencies from the `conda-forge` channel. Now, you are ready to create the `frame` module.

```
conda install fire -c conda-forge -y
```

### Frame Module

We are going to format our date and time information so that it appears inside a "frame" with a title. Inside the frame, the individual pieces of information reside on different lines. It should look something like this:



Start by importing the date module as well as the `fire` package. Use the conditional that checks if the script is being run directly and bind the `main` function (we will create this in a bit) to the `Fire` class.

```
import date
import fire

if __name__ == '__main__':
    fire.Fire(main)
```

Declare the `main` function that has the argument `day` with the default value of `today`. The variable `width` represents the size of the "frame". The `line` variable is a series of dashes that run the width of the frame. Call the `get_info()` method from the `date` module and store the dictionary as `info`. Print the "top" of the frame. Call the `display_date` function, which we will create in a bit. Finally, print the "bottom" of the frame.

```
def main(days='today'):
    width = 25
    line = width * '-'
    info = date.get_info(day)
    print(line)
    display_date(day, width, info)
    print(line)
```

The `display_date` function takes the day (yesterday, today, or tomorrow), the width of the frame, and the date and time information. Call the `display_title` function. This function prints the correct title dependent upon the value of `day`. The `center_text` function will center any text we pass it inside the frame. Call this function for the day of the week, the date and month (both on the same line), and the year. Only display the time if `day` is `"today"`.

```
def display_date(day, width, info):
    display_title(day, width)
    center_text(width, info['day'])
    center_text(width, f'{info["date"]} {info["month"]}')
    center_text(width, info['year'])
    if day == 'today':
        center_text(width, info['time'])
```

The `display_title` function takes `day` and `width` as arguments. Call the `center_text` function the appropriate title, verb, and number of dashes (which underline the title).

```
def display_title(day, width):
    if day == 'tomorrow':
        title = 'Tomorrow Will Be:'
    elif day == 'yesterday':
        title = 'Yesterday Was:'
    else:
        title = 'Today Is:'
    line = len(title) * '-'
    center_text(width, title)
    center_text(width, line)
```

Finally, the `center_text` function takes the width of the frame and the text to print. This function is going to print a `|` at the beginning and the end. So subtract 2 from `width` as this is the remaining blank space in which the text will be centered. Use the `center()` string method to center the text. Pass it `width - 2` so it can calculate the number of spaces to place on either side of the text.

```
def center_text(width, txt):
    print(f'|{txt.center(width - 2)}|')
```

### Testing the Script

Run the `frame.py` script. Do not pass it any arguments. By default it should return the information for today, including the time.

```
python frame.py
```

Try these variations:

- Test the script with `"yesterday"`. You should see yesterday's information without the time.

python frame.py --day 'yesterday'
- Test the script with `"tomorrow"`. You should see tomorrow's information without the time.

python frame.py --day 'tomorrow'

#### Code

Deactivate the `day` virtual environment.

```
conda deactivate
```

Reading Question

How does Pyinstaller allow its executables to run on platforms without Python?

☒ Pyinstaller bundles a Python interpreter with the executable.

☐ A Pyinstaller executable automatically downloads the latest version of Python before running.

☐ A Pyinstaller executable cannot be run on platforms without Python already installed.

☐ A Pyinstaller executable is run in the cloud where there is a Python interpreter.

Pyinstaller bundles a Python interpreter with the executable. This is how executables can run on platforms that do not have Python installed — the executable has its own interpreter.

Check It

#### Next